

PPT-LAT-Roadmap

Project: shannon-prime-lattice **Document role:** Operational roadmap. Read by every future session before doing work. **Status:** Living document. Mutable. Papers are scaffolding, not artefacts. **Last rewrite:** 2026-05-21 **Authors:** Knack + Claude + Gemini (Shannon-Prime team)

0. Read-this-first

This document supersedes the prior linear v1 roadmap that staged work as KSTE → engine → multi-node. That framing was too narrow. The project now spans four parallel engine backends, four (and growing) model families, inline weight and KV-cache compression as **foundational** (not an opt-in feature), and the Lattice blockchain track running in parallel with the inference work.

Two binding rules govern every session that picks up this roadmap:

1. **Anti-contamination rule.** Do **not** read, copy, vendor, or reach into source files under D:\F\shannon-prime-repos\shannon-prime\ or D:\F\shannon-prime-repos\shannon-prime-engine\. Those repositories exist for archival reference only. The math papers under D:\F\shannon-prime-repos\papers\PPT-ARM\ are **conceptual reference** — read them for theory, never paste code from them. This project rebuilds everything from scratch. Cross-pollination has already produced regressions in this organization at least ten times in six months. The memory item `feedback_no_cross_contamination` is load-bearing here.
2. **Papers are scaffolding, not artefacts.** PPT-LAT-Theory.md, PPT-LAT-Systems.md, and this file are the lattice from which the system is built, not finished publications. Sessions may amend any of them when reality contradicts the design. The blockchain protocol spec in particular is expected to evolve.

If those two rules feel restrictive, that is the point. The previous sessions that ignored them produced the artefacts collected in the `feedback_no_cross_contamination` and `feedback_dont_frankenpatch` memory items.

1. Why the scope expanded

The original v1 roadmap presumed a single hot path: build KSTE, wire it into one engine, demonstrate dedup, layer blockchain on top. Three realities forced the expansion:

- **Backend reality.** Real users sit on RTX 3000/4000, on Apple Silicon via Vulkan-MoltenVK, on Snapdragon phones with Hexagon DSP and HTP V69, and on Linux servers with AVX-512. A single-backend prototype cannot be promoted to a production system. Each backend has its own build environment, kernel idioms, and bring-up bugs. The roadmap treats them as four parallel tracks because trying to serialize them costs months for no architectural gain.
- **Model-family reality.** Llama 3.x, Qwen3.0–3.7, Gemma 2.5/3/4, and DeepSeek V4 do not share enough surface area to let one model carry the lattice features alone. They differ in RoPE shape, attention grouping, FFN topology (dense vs MoE), normalisation, and tokenisation. The roadmap therefore plans for an explicit model-family × backend matrix (a 7-row × 4-column grid in Phase 3), and assumes that filling this matrix is the project's centre of mass.

- **Compression-foundation reality.** Inline Q8 weight storage with Frobenius scale, Q4 mixed-precision under calibration, and VHT2 + Möbius + Spinor KV-cache compression are not “features that may be enabled later.” They are the **load-bearing memory layout** that makes long-context, multi-model, multi-backend inference fit at all. The roadmap treats compression as bedrock, not topping.

The Lattice features themselves — KSTE dedup, ARM gossip aggregation, dominance verification, two-token economy, blockchain protocol — layer on top of the compressed-inference stack and are off-by-default behind environment-variable gates until they are individually proven.

2. Phase summary table

Phase	Title	Goal in one line	Gate (must pass before advancing)	Wall-clock estimate
0	Bootstrap	Repos, papers, env scripts, workspace	DONE	DONE
1	Math core foundation	O_K, CRT-NTT, poly-ring, VHT2, Frobenius, KSTE built and tested in isolation	All T_* unit tests green on Win+Linux	3-4 weeks
2-CPU	Engine, CPU backend	Reference forward pass + compressed weights + NTT attention on x86	E_CPU_1..E_CPU_6 green	CLOSED 2026-05-22
2-CU	Engine, CUDA backend	Same scope as 2-CPU on NVIDIA GPU	E_CU_1..E_CU_6 green	CLOSED 2026-05-22
2-VK	Engine, Vulkan backend	Same scope as 2-CPU on cross-platform GPU	E_VK_1..E_VK_6 green	CLOSED 2026-05-23
2-HX	Engine, Hexagon backend	Same scope as 2-CPU on Snapdragon HTP V69	E_HX_1..E_HX_6 green	ESSENTIALLY CLOSED 2026-05-23 (formal tag pending E_HX_5/E_HX_6) CLOSED 2026-05-23
2-FMT	Engine, .sp-model on-disk format	Loader + transcoder + round-trip gate	E_FMT_1..E_FMT_4 green	

Phase	Title	Goal in one line	Gate (must pass before advancing)	Wall-clock estimate
2-L1	L1 ABI implementation in math-core	RELOCATE → VALIDATE → HANDLE → SESSION	Each sub-phase has its own gate; umbrella lat-phase-2-l1-closed	RELOCATE done; VALIDATE next
3	Model-family expansion	All four backends host all seven model families	$M \times B$ matrix green	8-12 weeks
4	Inline cache compression validated	PPL drift and memory savings measured per backend $\times$ model	Drift $\leq 1\%$ on calibrated families	4 weeks
4-MTP	Multi-token prediction (speculative decoding)	Transactional Spinor blocks + draft/verify/rewind via frozen L1 ABI primitives	M_MTP_1 : bit-identical output + $> 1.5 \times$ t/s speedup on code-heavy prompts at $K=4$	3 weeks; blocked by Phase 3 close
5	Lattice features (sieve, ARM, dominance)	Off-by-default ENV-gated overlays	Regression suite green when gates off	6 weeks
6	Two-node CRT-sharded inference demo	Dual-prime forward pass across two boxes	End-to-end demo run	3 weeks
7	DHT crawler skeleton	Kademlia-like routing, skeleton single-node first	Two-node lookup works	3 weeks

Phase	Title	Goal in one line	Gate (must pass before advancing)	Wall-clock estimate
8	Position-2-axis as-Arithmetic crawl as-signment + Fibonacci-Prime DHT	2-axis prime-lattice (semantic) $\times$ ϕ -hashed (load) key space	Deterministic re-assignment + uniform load under skewed inputs	2 weeks + 1 day (Fibonacci hashing)
9	ARM gossip aggregation + Golden-Ratio key init	Capacity-bounded HRR merge across peers; ϕ -spaced phases replace random projection	Capacity curve extends past prior K=64 ceiling	3 weeks + 2 days (key init)
10	Verification layer	Spot-check via $\leq_d$, slashing simulator	Slashing simulator stable	4 weeks
11	Two-token economy simulator	Discovery vs work-token dynamics	Equilibrium observed in simulator	3 weeks
12	Blockchain protocol design	Protocol scaffolding; simulator-only first	Simulator passes 1k-block run	6 weeks
13	End-to-end pilot	3 nodes, full stack, real metrics	Pilot report delivered	4 weeks

Wall-clock estimates assume one solo developer plus AI agents working a standard week. They are **planning numbers**, not commitments. The phase gates are non-negotiable. The estimates are.

2.1 How to read the phase table

A few notes on interpreting the table above for sessions picking up the project cold:

- The phases are not strictly serial. Phase 2's four backends are four parallel tracks. Phase 3's matrix is up to 28 parallel cells. Sections 3 and 20 below detail which work can overlap.
- A **gate** is a binary go/no-go check. If a gate fails, the phase is not closed and no later phase that lists it as a dependency may start. The estimates are not gates and may slip without changing the go/no-go status.
- The phase numbering survives reorganisation. If a future session decides Phase 8 should split into 8a and 8b, the table grows vertically; existing phase numbers do not shift. This

is what makes the `lat-phase-<n>-closed` tags durable.

2.2 The big picture in one paragraph

The project rebuilds the Shannon-Prime mathematical machinery from scratch (Phase 1), wires it into four engine backends (Phase 2), spreads it across the in-scope model families (Phase 3), validates inline compression end-to-end (Phase 4), layers the Lattice-specific content-dedup features on top behind environment gates (Phase 5), demonstrates two-machine sharded inference (Phase 6), grows the distributed-system layer (Phases 7–9), adds verification and economy simulation (Phases 10–11), specifies the protocol (Phase 12), and runs a three-node pilot (Phase 13). Every phase has a contract; sessions that close a phase write an offload note and tag the repository.

3. Anti-contamination, contracts, and offload pattern

3.1 Anti-contamination

Every Phase 1 file is created from a blank buffer. The math papers under `papers/PPT-ARM/` may be consulted for theorem statements; their implementations may not be copied. Sessions that violate this rule have historically produced (a) silent format drift between the new repo and the legacy one, (b) duplicate names that confuse later searches, and (c) regressions when a “shared” file in the legacy repo is updated and the new repo silently inherits a behaviour change.

If a session feels the urge to look at the legacy code “just for reference,” the correct move is to read the theorem statement in the paper and re-derive the implementation from scratch. The derivations are short. The bugs that come from cross-contamination are long.

3.2 Contract system

Every phase below lists:

- **Goal** — one paragraph.
- **Deliverables (file list)** — every file the phase must produce.
- **Tests (named, gated)** — every test the phase must pass, named so that future sessions can search for the exact identifier.
- **Entry conditions** — what must be done before starting.
- **Exit conditions** — what must be true to close the phase.
- **Dependencies** — which other phases gate this one.
- **Notes for the picking-up session** — what to read first, what gotchas past sessions have hit.

A phase is closed only when every deliverable exists, every test passes on both Windows and Linux (where applicable), and an offload note has been written.

3.3 Offload pattern

Each working session ends by writing `papers/SESSION-STATE-lat-<phase>.md` with: current phase, files touched, tests passing, tests failing, next concrete action, any open questions. Sessions that pick up the project read this file first, then this roadmap.

When a phase closes, the corresponding session-state file is renamed to `SESSION-CLOSED-lat-<phase>.md` and a one-paragraph summary is added to this roadmap’s “Phase log” appendix.

3.4 VSCode workspace + build environments

The workspace at `D:\F\shannon-prime-repos\shannon-prime-lattice.code-workspace` opens the four repositories the project actually touches:

- `shannon-prime-system` — math primitives.
- `shannon-prime-system-engine` — backend implementations.
- `shannon-prime-lattice` — Lattice features + this roadmap.
- `shannon-prime-lattice-node` — DHT / blockchain node code (later phases).

Build environment scripts live under `shannon-prime-system-engine/scripts/env/`:

- `env-cpu-msvc.bat` — Windows MSVC 19.x, AVX2/AVX512 flags pinned.
- `env-cpu-gcc.sh` — Linux gcc 12+, `-march=native` baseline.
- `env-cuda.bat` — VS2019 Build Tools + CUDA 12.4, `--use-local-env` flag, target SM 75/86/89.
- `env-vulkan.bat` — Vulkan SDK 1.3.x + `glslc`.
- `env-hexagon.bat` — Hexagon SDK 5.x on Windows host, `Git sh.exe` prepended to `PATH` per the `reference_hexagon_build_recipe` memory.

Versions are **pinned** in each script. A session that needs to bump a toolchain bumps the script in a dedicated commit and notes the change in the offload file.

3.5 GitHub workflow

All four repositories are private. Workflow is per-phase push, no pull requests required (solo developer plus agents). Tags are cut at each phase close, named `lat-phase-<n>-closed`. Commits inside a phase use the prefix `[lat-<phase>]` so the log is grep-able.

3.6 Testing discipline

Every phase keeps the **full regression suite green**, not just its own tests. This is the rule that prevents Phase 5+ Lattice features from silently breaking Phase 2 backends. Sessions land work behind a flag if they cannot make the regressions hold; turning the flag on is a separate commit that runs the suite again.

3.7 Platform-gate policy (amended 2026-05-21)

The original Phase 1 exit conditions read “all tests pass on Windows MSVC and Linux gcc.” That conflates three distinct platforms with three distinct costs, and as written it could not be satisfied in a single session on the current dev host (Windows, no local Linux, MSVC needs a separate toolchain bring-up). This amendment splits the platform gate explicitly. It does **not** weaken any test — every `T_*` still has to pass everywhere eventually; it only stages *which platform closes when*, and names the follow-up wave so it cannot be silently dropped.

The dev host has MinGW **gcc 15.2** and **MSVC v142 (VS2019 BT)** available; **no local Linux**.

Three-tier close, per Phase 1 subphase:

- **Tier 1 — Windows MinGW-gcc (closes in-session).** Every `T_*` test passes under MinGW gcc on Windows. This is the primary gate a Phase 1 session closes. Repo tag suffix: `lat-phase-1<X>-gcc-closed`.
- **Tier 2 — Linux gcc (CI).** Verified by the GitHub Actions workflow `.github/workflows/ci.yml` added in the Phase-1 scaffold pass (ubuntu-latest, gcc, cmake -G Ninja + ctest). Linux is not run locally; the CI run on push is the Linux gate. A subphase is not fully closed until its CI job is green on origin/main.

- **Tier 3 — Windows MSVC (follow-up wave).** A dedicated later session configures each module in a second build dir via `vcvarsall x64` and runs `ctest` under MSVC. Repo tag (full close): `lat-phase-1<X>-closed`.

__int128 and the cross-platform identity tests. The CRT-NTT parity oracle `ntt_ref_int128.c` uses `__int128`, which MinGW gcc and Linux gcc support but **MSVC does not**. The production path (`ntt crt.c`) is already forbidden `__int128` by `T_NTT_5`, so this only affects the oracle. Strategy: the oracle runs under gcc and dumps its reference vectors to a checked-in binary fixture (`core/ntt crt/ntt_ref_vectors.bin`); the MSVC build of the test loads that fixture and compares bytes, rather than recompiling the `__int128` oracle. The same checked-in-fixture pattern serves the other cross-platform identity gates (`T_NTT_2/3`, `T_VHT_5`, `T_KSTE_4`): the gcc build is the byte-source-of-truth, MSVC and Linux CI compare against it. This makes “bit-identical across platforms” a concrete file-diff rather than a re-derivation.

So for a Phase 1 session on this host, “subphase closed” means **Tier 1 green locally + Tier 2 green in CI**, tagged `-gcc-closed`. Tier 3 (MSVC) is tracked as open in the offload note and closed in its own wave.

4. The math primitives — what we are recreating

This section names every math primitive the project will rebuild from scratch and what each one is for. Implementations come in Phase 1.

4.1 O_K integer arithmetic over $Q(\sqrt{-163})$

$Q(\sqrt{-163})$ is one of the nine imaginary quadratic fields with class number 1 (a Heegner number). The ring of integers O_K is a unique factorisation domain. We choose $Q(\sqrt{-163})$ specifically because:

- UFD means every nonzero non-unit factors uniquely (up to units and order) into irreducibles. That property anchors Möbius reconstruction and lets us treat token indices as products of irreducibles without ambiguity.
- The class group is trivial. No nontrivial ideal class equivalences haunt the storage layout.
- The discriminant -163 fits comfortably in 64-bit arithmetic for the norms we care about.

All exact arithmetic — KSTE node values, Möbius reconstructions, ARM binding accumulators — happens in O_K . The implementation must support add, mul, conjugate, norm, division-with-remainder when the divisor’s norm permits, and irreducible factorisation up to a configurable norm bound.

4.2 Polynomial-ring attention over $R_q = Z_q[x]/(x^N+1)$

R_q is the negacyclic polynomial ring used by CKKS and most modern lattice cryptosystems. We use it as a **replacement for the softmax attention kernel**: queries, keys, and values are encoded as polynomials in R_q ; the inner-product step becomes a polynomial multiplication modulo $(x^N + 1)$; the softmax becomes a p-adic exponential on the coefficients followed by a normalisation pass.

Phase 1 targets $N = 256$ (matching the per-head dimension of the current model targets) and parameterises N over $\{128, 256, 512\}$ so that all three sizes share a single code path. ($N = 1024$ was in the original draft but is **not supported** on the frozen CRT primes — see §4.3; the primes admit a primitive $2N$ -th root only up to $N = 512$. Amended 2026-05-21.)

4.3 CRT-NTT dual-prime kernel with no __int128

Multiplying in R_q efficiently requires the Number Theoretic Transform. We use a dual-prime CRT decomposition so the kernel never needs 128-bit arithmetic and ports to any 64-bit ALU including Hexagon and Vulkan compute shaders.

- $q_1 = 1073738753$, $q_2 = 1073732609$ (two 30-bit Proth primes). These are **frozen** — the dominance-commitment verification and the DHT key topology (§4.4) derive from these exact prime residues, so they do not change.
- Each prime admits a primitive $2N$ -th root of unity for **N up to 512**, not 1024. Both have $q-1 = 2^{10} \cdot (\text{odd})$, so the 2-adic valuation is 10: $2N \mid (q-1)$ holds for $2N \leq 1024$, i.e. $N \leq 512$. A negacyclic NTT at $N = 1024$ would need $2^{11} \mid (q-1)$, which neither prime satisfies. The supported ring degrees are therefore $\{128, 256, 512\}$. (Amended 2026-05-21 after the original draft over-claimed $N = 1024$.)
- The combined modulus $M = q_1 \cdot q_2 \approx 2^{60}$ carries the full result without overflow.
- CRT recombination uses Barrett reduction; no 128-bit divide.

Independent NTT branches run per prime. Bit-exactness is checked against a reference 60-bit __int128 implementation that exists **only as a parity oracle** (compiled out of production builds).

4.4 Frobenius lifting for Q8 / Q4 weight storage

Frobenius lifting takes a quantised weight stored as a small integer plus a per-row shift and decompresses it inline at matmul time. The per-row Frobenius scale is the key design choice: a single per-tensor shift collapses to noise on Q4 (the cautionary tale from prior validation), while per-row shifts hold accuracy.

- Q8: 1 byte per coefficient + per-row scale. 8× memory compression of the unquantised arena. Production target.
- Q4: 4 bits per coefficient + per-row scale + calibration table. Mixed-precision path; gated on a per-tensor calibration that may promote outlier rows to Q8.

Decompression is fused into the matmul kernels, not done as a separate pass. There is no decoded scratch arena in production builds.

4.5 VHT2 + Möbius reorder + 63-byte Spinor block

The KV cache is stored as a sequence of 63-byte Spinor blocks. Each block carries:

- A VHT2 (Vilenkin Hierarchical Transform) header that describes which basis vectors the block compresses.
- A Möbius-reordered coefficient body.
- A trailing checksum byte.

The byte layout is **frozen** the moment Phase 1D ships. Future sessions do not adjust it without a compatibility migration plan. The cache file format and the on-wire DHT format both reference this layout, so a silent change breaks every node on the network simultaneously.

4.6 KSTE encoder + Tier-0/Tier-1 dominance

The Knight-Spinor Tree Encoder maps a K-vector into a 64-byte packed tree in $T_{\{60,3\}}$. The encoder is deterministic, lossless under the dominance partial order, and produces a compact signature suitable for hashing.

Tier-0 dominance compares roots. Tier-1 dominance compares the first level of children. The Lattice dedup mechanism uses Tier-0 to bucket content into rough equivalence classes and

Tier-1 to confirm a match before counting it as a duplicate. The encoder must produce identical output on every backend so that two nodes can compare signatures without a tie-break protocol.

4.7 ARM — Algebraic Resonance Memory

ARM stores HRR-style key-value bindings in the CRT cyclotomic ring. Binding is a polynomial multiplication; recall is an inverse-binding multiplication followed by a similarity search. The choice of the CRT cyclotomic ring is what lets ARM share its kernel inventory with the poly-ring attention layer: a single NTT-based pointwise multiply serves both. Storing ARM bindings as 63-byte Spinor blocks would in principle work too; in practice we store the polynomial coefficients directly because ARM banks are written-once-read-many and benefit from a flat layout.

Phase 1 ships ARM as a single-node primitive. Phase 9 layers gossip on top so peers can merge their ARM banks with bounded capacity loss. The capacity bound is the usual HRR $\sqrt{K}$ bound: with K bindings packed into a single ring element of dimension N , recall accuracy degrades as roughly $\sqrt{K/N}$. For $N = 256$ this means a slab holds about 16 high-confidence bindings before recall fidelity drops below 0.9 cosine similarity, which is the threshold the Phase 9 capacity-curve experiment will report against.

4.8 Inline weight compression as a foundation, not a feature

The single most consequential design choice in this roadmap is that inline Q8 (and eventually Q4) weight storage is not an optional optimisation but the **default memory layout**. Every backend’s matmul kernel reads packed bytes and applies the per-row Frobenius scale inline; there is no decoded fp32 arena that holds the same weights in expanded form. The reason this matters is twofold:

- Memory pressure. An unquantised arena on Gemma3-1B already costs 10.4 GB; the Q8 packed form is 1.3 GB. The unquantised arena does not fit on the phone path at all, and it costs 8× the network transfer on the multi-node CRT-shard path.
- Bandwidth determinism. Every backend uses the same packed byte layout, so cross-backend verification compares bytes for bytes rather than fp32 approximations. This is what makes the cross-backend determinism test (OQ-8) tractable at all.

The cost of treating compression as foundational is that every backend must implement the packed-byte matmul before any larger model can be brought up. Sessions that attempt to “do correctness first, compression later” hit a wall when the unquantised path runs out of memory on the test rigs and need to backtrack to compression-first anyway.

4.9 Inline KV-cache compression as a foundation

The same logic applies to the KV cache. The Spinor block layout specified in §4.5 is the only KV layout in the production paths. There is no “uncompressed KV cache” mode in production builds; the reference fp32 cache exists for the parity tests only, compiled out of release builds. The reason is identical to the weight-compression case: long-context inference does not fit on any of the target backends without KV compression, and the CRT-shard and DHT paths both consume the on-disk and on-wire byte layout directly, so the production format has to match from day one.

5. The 13-step Prime Power Transformer (canonical table)

The replacements below are the canonical algebraic interpretation of every step in a transformer forward pass under the PPT framework. They are reproduced verbatim from Paper I §10 so future sessions can grep for the exact wording.

Step	Operation	Algebraic replacement
1	Embedding lookup	Möbius reconstruction over squarefree token indices; CRT vocabulary sharding
2	RMSNorm (pre-attn)	Mersenne-prime scaling; Poncelet closure $d^2=R^2-2Rr$
3	Q/K/V projections	Twin-prime head pairing; sexy-prime 6:1 GQA grouping
4	SP Write (KV → archive)	Poncelet closure as eviction trigger; CRT-sharded KV
5	FUSED_KQ	UFD-exact decompression; Heegner endomorphism
6	Softmax	p-adic exponential on integers; circulant attention on closed orbits
7	Fused V weighted sum	Spinor reconstruction across twin-paired heads
8	Attention output projection	CRT decomposition of W_O into independent sub-matrices
9	FFN (skeleton + residual)	Mersenne-dimensional skeletons; n^2+n+41 cold-start
10	Activation oracle update	Cramér prime-gap prefetch; Poncelet early exit
11	Residual add + norm	Group-law residual on $E(K)$
12	Per-layer loop	$n\delta\equiv 0$ adaptive depth; caustic projection
13	LM head	CRT pruning of vocabulary logits; Mersenne-prime sampling

This is the operational mapping from “what the model does” to “what we build.” Every Phase 2 backend ultimately wires these thirteen steps into its forward pass. Every Phase 3 model family threads the same table through its specific topology.

6. Theorems and extensions on the anchor

Treat the following as the “already proven” set. Sessions do not need to re-prove them; they need to call out when an implementation contradicts them.

- **T1 — Endomorphism realization.** A hidden-state trajectory through L transformer layers embeds in the L -fold product of an elliptic curve E over O_K . Layer composition is endomorphism composition.
- **T2 — Möbius UFD compression.** Exact reconstruction over O_K is possible at the squarefree basis. This is the theoretical anchor for the squarefree token index in step 1.
- **T3 — Hasse-Weil = Shannon limit.** The Hasse-Weil bound $|E_p(F_p) - (p+1)| \leq 2\sqrt{p}$ coincides with the Shannon-channel capacity bound applied to the CM curve. The two limits are the same constant to within a unit.
- **T4 — Frobenius cancellation.** On Gemma3-1B, the Frobenius lift is bit-identical to six significant figures. The original validation produced PPL 13.11 against a baseline of 13.12

(delta 0.08%). That single-number figure was the original acceptance criterion; under the production **per-row** Frobenius arena (E_CPU_9, ~1% lossy by design) it is **superseded** by the split T_FRO_4 gate of §8.2 — forward correctness (engine-f32 vs the f16 oracle $\leq 0.05\%$) separated from per-row Q8 quality (drift vs engine-f32 $\leq 2\%$). See the §8.2 T_FRO_4 closure clause.

- **T5 — Deuring / CM Sato-Tate.** The Frobenius trace a_p is asymmetrically distributed between primes that split and primes that remain inert in O_K . This asymmetry is what makes class-number-1 fields work for compression — the inert primes carry more bits per symbol.
- **T6 — CRT exact sharding.** The dual-prime kernel is bit-identical to the 60-bit reference on Linux gcc and Windows MSVC. Portability across any 64-bit ALU is implied.
- **E9.1 — Stern-Brocot RoPE.** Discrepancy $\phi = 0.00134$ against 0.05576 for standard RoPE. The Stern-Brocot positional encoding is a strict improvement on quasirandomness grounds.
- **E9.2 — Weil pairing on $E[n]$.** Miller’s algorithm validated; bilinearity confirmed empirically. The pairing math is solid; the open research question is the embedding from hidden states into $E[n]$.
- **E9.3 — Hecke multiplicativity.** 20 of 20 random trials confirm the Hecke operator multiplicativity at composite levels.
- **E9.5 — LLL reduction.** 20 of 20 trials confirm LLL-reduced bases produce smaller KV-write footprints than the naive basis.
- **E9.6 — BSD analytic rank.** Toy curves verified via Sage. Not load-bearing for the production system; included for completeness.
- **E10 — Iwasawa $\mu=0$.** Residual-stream depth stability follows from the $\mu=0$ conjecture in the relevant Z_p -extension. The empirical evidence is consistent with $\mu=0$ on all curves tested.

When a Phase 2 backend produces a result inconsistent with one of these theorems, the theorem is correct and the implementation is wrong. That is the working assumption until proven otherwise.

The reason this rule reads so strongly is operational: every time a prior session has assumed the math was off, the math has turned out to be right and the bug has been in the storage layout, the kernel ordering, or a sign convention. The theorems above have been validated to six significant figures on real hardware running real models; the chance that a fresh backend implementation is the first place those theorems fail is vanishingly small compared to the chance that the implementation has a transposed index somewhere. The corollary is that when a backend disagrees with another, the older backend is the ground truth and the newer one is the suspect.

A second corollary: do not bend the math to make the implementation easier. There is a recurring temptation, especially on the Phase 2-HX track, to drop the dual-prime CRT in favour of a single 30-bit prime because “the test corpus fits anyway.” Resisting that temptation is the whole reason Phase 1B’s T_NTT_5 grep gate exists. The dual-prime kernel is not an optimisation. It is the substrate Phase 6’s two-node demo runs on, the substrate Phase 8’s position-as-arithmetic key derivation reads from, and the substrate the verification layer audits. Collapsing it to single-prime to save bring-up time costs multiple later phases.

7. Phase 1 — Math core foundations

Goal. Stand up every math primitive in section 4 as a standalone library inside shannon-prime-system, with deterministic unit tests that run on Windows MSVC and Linux gcc.

Dependencies. Phase 0 (DONE).

Subphases run in parallel. Phase 1A through 1F are independent modules. A session can pick any one without blocking the others. The recommended priority order — for a serial reader — is 1A → 1B → 1D → 1C → 1E → 1F, because the polynomial-ring and Frobenius layers benefit from the CRT-NTT kernel being already in place, and KSTE wants the VHT2 block format frozen first.

Why these particular primitives. The Phase 1 list is not a buffet of mathematical curiosities. Each primitive answers a specific question about how the production system stores and moves bits:

- **O_K** answers “how do we factor token indices uniquely so Möbius reconstruction works without ambiguity?” The class-number-1 property is the load-bearing piece.
- The dual-prime CRT-NTT answers “how do we multiply in the cyclotomic ring on every backend without needing 128-bit arithmetic?”
- Polynomial-ring attention answers “how do we replace softmax with something that lives in the same algebraic universe as the cache representation, so the kernel inventory shrinks?”
- The Spinor block answers “what is the on-disk and on-wire byte layout of a single KV record?” The 63-byte frozen layout is the contract.
- Frobenius lifting answers “how do we store weights at 1 byte per coefficient and still get bit-identical PPL?”
- KSTE answers “how do we produce a content signature that two nodes can compare bit-for-bit without a tie-break protocol?”

Implementing all six in parallel is what makes Phase 1 fit in a few weeks rather than a few months. Sessions that try to serialise will discover that the bottleneck is rarely arithmetic — it is build environments, harness wiring, and offload notes. Spreading the implementation work across parallel subphases keeps any single bottleneck from gating the whole phase.

7.1 Phase 1A — O_K arithmetic over $\mathbb{Q}(\sqrt{-163})$

- **Deliverables.** `system/ok/ok_int.h`, `ok_int.c`, `ok_int_test.c`.
- **Tests.**
 - `T_OK_1` — UFD verification on 256 random norms $\leq 2^{20}$.
 - `T_OK_2` — Norm and conjugate identities ($N(\alpha\beta) = N(\alpha)N(\beta)$, $\text{conj}(\text{conj}(\alpha)) = \alpha$).
 - `T_OK_3` — Multiplication closure (every product lands back in `O_K` with correct integer coordinates).
 - `T_OK_4` — Ideal class triviality: every fractional ideal of norm $\leq 2^{16}$ is principal.
 - `T_OK_5` — Round-trip from rational integer to `O_K` and back.
 - `T_OK_6` — Irreducible factorisation for 1024 random norms $\leq 2^{14}$.
- **Entry conditions.** None.
- **Exit conditions.** All six tests pass on Win+Linux. No undefined behaviour under UBSan.
- **Notes.** The Heegner number 163 is non-negotiable. Other Heegner numbers exist (1, 2, 3, 7, 11, 19, 43, 67, 163) but only 163 has the discriminant we want.

7.2 Phase 1B — CRT-NTT dual-prime kernel

- **Deliverables.** `system/ntt/ntt_crt.h`, `ntt_crt.c`, `ntt_crt_test.c`, `ntt_ref_int128.c` (parity oracle, compiled out of production).
- **Tests.**
 - `T_NTT_1` — Forward/inverse round-trip on 4096 random polynomials at each $N \in \{128, 256, 512\}$.
 - `T_NTT_2` — Bit-exactness vs `ntt_ref_int128` on Linux gcc.
 - `T_NTT_3` — Bit-exactness vs `ntt_ref_int128` on Windows MSVC.

- T_NTT_4 — Pointwise multiply followed by inverse equals coefficient-wise modular multiplication of the original polynomials (negacyclic convolution).
- T_NTT_5 — No 128-bit type used in the production path (static assert in header, plus a grep gate in CI).
- **Entry conditions.** None.
- **Exit conditions.** All five tests pass.
- **Notes.** Barrett reduction for the CRT step. Twiddle factors precomputed and cached per N. The parity oracle is a regression anchor for the lifetime of the project — keep it building even when it is not in the production path.

7.3 Phase 1C — Polynomial-ring attention R_q

- **Deliverables.** system/poly/poly_ring.h, poly_ring.c, poly_ring_test.c.
- **Tests.**
 - T_PR_1 — Polynomial multiply matches schoolbook reference at $N \in \{128, 256, 512\}$ for 1024 random polynomial pairs each.
 - T_PR_2 — Inner-product attention $KL \leq 1e-7$ versus softmax baseline at $d = 256$ (matches the prior Gemma3 head-size result on the legacy repo).
 - T_PR_3 — Negacyclic property: $x \cdot p(x)$ wraps as expected (the last coefficient becomes the negated leading coefficient).
 - T_PR_4 — Cross-N stability: the same input expressed at $N=256$ and $N=512$ (zero-padded) produces the same result up to the shared coefficient prefix.
- **Entry conditions.** Phase 1B closed.
- **Exit conditions.** All four tests pass.
- **Notes.** The KL bound T_PR_2 is the closest thing the project has to a soft acceptance criterion for “the math is right.” Take it seriously.

7.4 Phase 1D — VHT2 + Möbius reorder + 63-byte Spinor block

- **Deliverables.** system/spinor/spinor_block.h, spinor_block.c, vht2.c, möbius_reorder.c, spinor_test.c. Frozen byte layout specified in papers/PPT-LAT-Systems.md §3.
- **Tests.**
 - T_VHT_1 — Block round-trip: encode then decode identical bytes for 65,536 random vectors.
 - T_VHT_2 — Möbius reorder bijection: every permutation is a true permutation (no element collisions).
 - T_VHT_3 — 63-byte total size verified by `sizeof(spinor_block_t)`.
 - T_VHT_4 — Checksum byte detects any 1-bit corruption in the body.
 - T_VHT_5 — Endianness: identical bytes produced on little-endian Linux and Windows hosts.
 - T_VHT_6 — Layout frozen flag: a LAYOUT_VERSION constant in the header must be incremented by hand if any field moves, and CI refuses a layout change without a migration plan file.
- **Entry conditions.** None.
- **Exit conditions.** All six tests pass; layout reviewed and signed off in the offload note.
- **Notes.** The 63-byte total is deliberate — 64 minus the checksum, packed tight, no padding. Future sessions: do not add a “convenient” reserved byte.

7.5 Phase 1E — Frobenius lift for Q8 weight storage

- **Deliverables.** system/frobenius/frobenius_lift.h, frobenius_lift.c, frobenius_test.c.
- **Tests.**

- T_FRO_1 — Per-row scale picker correctness against the ceiling-shift reference (which exists from prior validation; reimplemented from scratch under the anti-contamination rule).
- T_FRO_2 — Encode-then-decode round-trip max error ≤ 1 ULP for a fixed test matrix.
- T_FRO_3 — 8× memory reduction confirmed on a 4096×4096 weight matrix.
- T_FRO_4 — On Gemma3-1B, Frobenius-lifted weights produce PPL matching the f16 oracle (the T4 acceptance check repeated under new code). **DEFERRED to Phase 2** (amended 2026-05-21): T_FRO_4 requires a working forward pass + a loaded Gemma3-1B, neither of which exists in Phase 1 (the engine lands in Phase 2). **CLOSED in Phase 2-CPU 2026-05-22** via the split gate (forward-correctness $\leq 0.05\%$ vs oracle; per-row Q8 drift $\leq 2\%$) — see the §8.2 closure clause; the original single “within 0.1%” target was superseded because the production arena is per-row ($\sim 1\%$ lossy). Phase 1E closes on the pure-math tests T_FRO_1..3.
- **Entry conditions.** Phase 1A closed.
- **Exit conditions.** T_FRO_1..3 pass under UBSan; T_FRO_4 deferred to Phase 2 (now CLOSED there 2026-05-22 via the split gate — see the §8.2 closure clause).
- **Notes.** Q4 is **not** in Phase 1E. Q4 lives in Phase 4 because it needs calibration tables that depend on a working forward pass.

7.6 Phase 1F — KSTE encoder + Tier-0/Tier-1 dominance

- **Deliverables.** system/kste/kste_encode.h, kste_encode.c, kste_dominance.c, kste_test.c.
- **Tests.**
 - T_KSTE_1 — Encoder determinism: identical K-vector produces identical bytes 1,000,000 trials.
 - T_KSTE_2 — Tier-0 dominance partial-order axioms: reflexivity, antisymmetry up to equivalence, transitivity.
 - T_KSTE_3 — Tier-1 confirmation rule: if two trees pass Tier-0 and Tier-1, they share the same first-level child set.
 - T_KSTE_4 — Cross-platform identity: encoder produces identical bytes on Windows MSVC and Linux gcc.
 - T_KSTE_5 — 64-byte packed-tree size enforced.
- **Entry conditions.** None.
- **Exit conditions.** All five tests pass.
- **Notes.** Tier-0/Tier-1 nomenclature comes from the original KSTE paper. Future sessions sometimes try to “simplify” to a single tier for ergonomic reasons; the two-tier check is what makes the Lattice dedup adversarially robust. Do not collapse it.

7.7 Phase 1 exit

Phase 1 closes when all six subphases close, the regression runner prints six green sections, and SESSION-CLOSED-lat-1.md is written. The repository tag is lat-phase-1-closed.

7.8 Pseudocode sketch — Phase 1B kernel skeleton

The following pseudocode captures the shape of the dual-prime CRT-NTT kernel without descending into a real implementation. It exists here to anchor the API a Phase 1B session is expected to land.

```
struct ntt_ctx {
    uint32_t N;           // ring degree, in {128, 256, 512}
    uint32_t q1, q2;      // two 30-bit Proth primes
    uint32_t* psi1; uint32_t* psi2; // 2N-th roots of unity per prime
```

```

    uint32_t* inv_psi1; uint32_t* inv_psi2;
    // Barrett mu values for each prime
};

ntt_ctx* ntt_init(uint32_t N);
void ntt_forward (const ntt_ctx*, const int32_t* in, uint32_t* out1, uint32_t* out2);
void ntt_inverse (const ntt_ctx*, const uint32_t* in1, const uint32_t* in2, int64_t* out);
void ntt_pointwise_mul(const ntt_ctx*, const uint32_t* a1, const uint32_t* a2,
                        const uint32_t* b1, const uint32_t* b2,
                        uint32_t* out1, uint32_t* out2);
void ntt_crt_recombine (const ntt_ctx*, const uint32_t* x1, const uint32_t* x2, int64_t* out);

```

The contract is: `ntt_forward` produces two residue vectors; `pointwise multiply` happens per residue (no cross-prime arithmetic); `ntt_inverse` applies the inverse transform per residue and `ntt_crt_recombine` reassembles a signed 60-bit result via Barrett reduction. No 128-bit type appears anywhere. The same kernel is reused by 1C (poly-ring attention), 4.7 (ARM), and 1E (Frobenius matmul) without modification.

7.9 Pseudocode sketch — Phase 1D Spinor block

```

#define LAYOUT_VERSION 1u // bumping requires migration plan
struct spinor_block_t { // 63 bytes total, packed
    uint8_t vht2_header[7];
    uint8_t mobius_body[55];
    uint8_t checksum; // CRC-8 of header || body
};
_Static_assert(sizeof(spinor_block_t) == 63, "frozen layout");

```

Encoders and decoders must round-trip on every backend; the cross-platform identity test `T_VHT_5` specifically compares bytes byte for byte between Linux gcc and Windows MSVC builds.

8. Phase 2 — Engine backend tracks (CPU, CUDA, Vulkan, Hexagon)

Goal. Each backend supports a single reference model end-to-end: GGUF load → forward pass → token-level output. Frobenius-lifted weights are inline-decompressed. NTT-attention is wired but the **sieve is OFF**. Inline KV-cache compression is wired but the **Lattice features are OFF**. The reference model is Qwen3-0.6B Q8.

Dependencies. Phase 1 closed.

Parallelism. The four backends are independent. Sessions can run 2-CPU and 2-CU concurrently with no shared state. 2-VK and 2-HX can start the moment their build environments are sorted, regardless of 2-CPU and 2-CU status. The matrix below shows the per-backend subphase plan — same letter-coded structure across all four.

8.1 Per-backend subphase template

For each backend $B \in \{\text{CPU, CU, VK, HX}\}$:

- **2-B.A — GGUF loader.** Parse the GGUF header, materialise tensors into backend-native storage. No SP transforms applied yet.

- **2-B.B — Forward pass on Qwen3-0.6B Q8.** Reference correctness vs llama.cpp logits on the same prompt. Acceptance: max-token logit difference $\leq 1e-4$ absolute or $\leq 0.1\%$ relative.
- **2-B.C — Frobenius-quantised matmul with inline decompression.** Production memory layout. 8× compression confirmed end-to-end.
- **2-B.D — Backend-specific vectorisation.** AVX2 for CPU, AVX512 optional second pass, CUDA tensor cores where applicable, Vulkan subgroup ops, HVX vectorisation for Hexagon.
- **2-B.E — NTT-attention path wired in.** Sieve OFF. PPL must remain within Phase 1C's T_PR_2 tolerance.
- **2-B.E.1 — Lossless Polynomial-Shift RoPE Cache (the production spec).** Precondition: SP_ENGINE_NTT_ATTN=1 (poly-ring attention active). Mechanism: inside the negacyclic cyclotomic ring $R_q = \mathbb{Z}_q[x]/(x^N + 1)$, a RoPE rotation by angle $\Delta \cdot \theta_d$ becomes a *discrete polynomial index shift modulo N* — no continuous trigonometry, no cos / sin table. The relative-attention rotation cache collapses from $O(\text{ctx} \cdot D)$ fp32 values to $O(\text{ctx})$ integer shift offsets. For $D = 128$, $\text{ctx} = 4096$: from $4096 \times 128 = 524,288$ floats (2 MB) down to 4096 int32 offsets (16 KB). **~128× memory reduction, mathematically lossless on stock geometric-RoPE models.** Applies to Gemma3, Qwen3, Llama 3.x and every other model the engine targets — no ϕ -RoPE precondition, no frequency-schedule swap, no PPL-drift trade.

Acceptance: T_PR_2 / E_CPU_5 already measures the gate. The NTT polynomial-shift representation is exact inside the ring up to int32 quantisation; E_CPU_5 on Qwen3-0.6B has it at $\text{KL} \approx 2.7 \times 10^{-10}$ mean against the fp32 reference. Activating the shift-cache representation does not introduce any additional error beyond what E_CPU_5 already gates. Per-backend gates inherit the same property: when the backend's NTT-attention path passes E_B_5 / T_PR_2, the polynomial-shift cache is in-spec by construction.

Why this works on stock RoPE. The Three-Gap Theorem (T7) is about *linear* irrational sequences $\{k\alpha\}$; geometric RoPE $\theta_d = \text{base}^{-2d/D}$ does not satisfy that precondition, so the original “ ≤ 3 distinct adjacent gaps across dimensions” framing did not apply. But the cyclotomic-ring representation bypasses the frequency-axis sort entirely — inside R_q , every rotation is already a discrete index permutation regardless of the underlying θ_d distribution. The win is structural to the ring, not to the frequency schedule.

Gate: built into SP_ENGINE_NTT_ATTN=1. When NTT-attention is active, the polynomial-shift cache is the production representation; when NTT-attention is off, the engine falls back to the fp32 rotation table. The previous SP_ROPE_THREE_STEP=1 / SP_ROPE_3STEP_CACHE=1 / SP_ROPE_PHI=1 gate proposals are retired in favour of NTT-coupling.

The ϕ -RoPE schedule swap and the Three-Gap-derived frequency-sort cache restructuring are moved to **\$20 Research Track** at the bottom of this document. They remain interesting for a future ϕ -RoPE-trained or fine-tuned model but are not on the Phase 2 critical path.

- **2-B.F — KSTE-encoded KV cache.** Gated by SP_KSTE_KV=1. Off by default. Phase 2 only requires that the encode path produces identical signatures across backends and that decode reproduces the same KV.

Tests E_B_1..E_B_6 mirror the six subphases. The numbering follows the subphase letter: E_CPU_1 covers 2-CPU.A, E_CPU_2 covers 2-CPU.B, and so on.

Amended 2026-05-22: the per-backend template is extended with the foundational-compression items first specified for CPU in §8.2.1 — **G: Q4 inline weight compression** (E_B_7), **H: inline VHT2+Spinor KV codec** (E_B_8), and the **persistent-KV O(n) decode loop** (GEN_B). Each later backend (2-CU/VK/HX) mirrors E_B_7/E_B_8/GEN_B alongside E_B_1..E_B_6, with the same gates (SP_ENGINE_FROB=3, SP_KV_SPINOR=1, qwen3_generate_kv) defaulting OFF and the cross-backend tolerance (output vs CPU within 1e-3). Do not silently drop them. See §8.2.1 for the CPU definitions and gates.

Three-Gap deliverables across phases (added on the T7 anchor in PPT-LAT-Theory): the Three-Gap Theorem (T7) unifies four optimisations that land in different phases of the roadmap but share one substrate (golden-ratio rotation in phase space). Cross-reference summary so a session picking up any one of these knows where the others live:

- **2-B.E.1 — Lossless Polynomial-Shift RoPE Cache** (production spec, all backends). RoPE rotation in the cyclotomic ring R_q is a discrete polynomial index shift mod N . Rel-attn cache collapses from $O(\text{ctx} \cdot D)$ fp32 values to $O(\text{ctx})$ int32 offsets — $\sim 128\times$ memory reduction. Gated by SP_ENGINE_NTT_ATTEN=1. Lossless on stock RoPE models; acceptance is the same E_B_5 / T_PR_2 gate that already closes the NTT-attention path (no separate gate needed).

(ϕ -RoPE frequency swap and the original Three-Gap frequency-sort cache restructuring are demoted to §20 Research Track.)

- **Phase 4 amendment — Fibonacci KV sub-sampling.** When KV cache exceeds memory bounds, retain tokens at positions $\lfloor k\varphi \cdot N \rfloor \bmod N$ instead of FIFO/LRU. Bounded discrepancy on the temporal axis. Gated by SP_KV_FIB=1. Validates against the PPL-drift sweep already run for VHT2+Spinor.
- **Phase 8 amendment — Fibonacci-Prime DHT** (1-day deliverable under Phase 8). 2-axis address space: prime-factored lattice for semantic adjacency (axis 1) $\times$ Fibonacci hashing for load balance (axis 2). Solves Kademia clustering natively without cryptographic hashing. See PPT-LAT-Systems §4.4.
- **Phase 9 amendment — ARM Golden-Ratio key init** (2-day deliverable under Phase 9). Replace random projection + Gram-Schmidt with deterministic φ -spaced phases in R_q . Capacity curve expected to extend past the prior K=64 ceiling at 0.15 cosine recall. See PPT-LAT-Systems §4.2.
- **§6.x amendment — Validator selection by Golden-Ratio rotation** (rolls into Phase 11/12 — Two-token economy and Blockchain). Replace VRF/randao beacon with deterministic $\{b\varphi\}$ partitioning of the stake-weighted validator set. Provably fair, no consensus rounds needed. See PPT-LAT-Systems §6.x.

All five items default OFF / additive. None are blocking dependencies for the foundational compression and model-family work — they are optimisations that layer on once the underlying compression and DHT infrastructure is green.

8.2 Phase 2-CPU (the canonical track)

STATUS: CLOSED 2026-05-22. All six E_CPU gates + T_FRO_4 split-gate green. Engine commits through 3431cf8 (SP2 41c5e19 SentencePiece encode/decode, SP3 3431cf8 sp_perplexity + test_ppl). T_FRO_4: engine-f32 vs f16 oracle -0.0146% (gate $\leq 0.05\%$); per-row Q8 arena drift -0.74% (gate $\leq 2\%$). Full CPU regression 20/20 green incl. T_FRO_4. Tag lat-phase-2-cpu-fro4-closed, lat-phase-2-closed. Offload: SESSION-CLOSED-lat-2-CPU.md.

T_FRO_4 amendment (2026-05-23, §8.7.5 supersession). The split-gate above ran at f32 working precision — the path the project is moving off of due to host-RAM ceilings on

production-context inference. Under §8.7.5 Phase 2-L1.FP16, T_FRO_4 is redefined:

- gate (a) → engine-**fp16** PPL vs f16 oracle $\leq 0.05\%$ (tighter than the f32 original by construction; same precision both sides).
- gate (b) → per-row Q4 mixed-precision cross-backend at fp16, bounded by fp16's intrinsic floor (expected zero by SP Frobenius-lift identity; see §8.8.1 generalization paragraph).

The -0.0146% / -0.7353% numbers above remain the historical record of the f32-working-precision pre-shift path. New runs under §8.7.5-and-beyond use the redefined gate. Section 8.7.5's E_FP16_1 deliverable lands the fp16-vs-oracle measurement; E_FP16_2 lands the cross-backend fp16 identity measurement.

Original spec, retained for historical context:

- **Build env.** scripts/env/env-cpu-msvc.bat (Windows) and scripts/env/env-cpu-gcc.sh (Linux).
- **Reference model.** Qwen3-0.6B Q8.
- **Tests.**
 - E_CPU_1 — GGUF loader round-trips header bytes.
 - E_CPU_2 — Forward pass reproduces the stock-llama.cpp **distribution** on identical token IDs (see §8.6.1 for why a per-logit tolerance is the wrong gate). Three properties, default pure-f32 path:
 - (a) top-1 argmax agrees at every position; (b) ggml's top-1 lies inside the engine's top-5 and vice versa at every position; (c) mean $KL(ggml \parallel engine)$ over positions $< 1e-5$ nats (measured $\sim 2.3e-6$ on Tier-1; override via SP_KL_MAX).
 - E_CPU_3 — Frobenius/Q8 weight path. (a) **Lift faithfulness** (the "identical logits to a reference fp32 matmul"): the inline-lift matmul (SP_ENGINE_FROB=1, accumulate $q \cdot x$ then scale once) and the dequant-then-f32-dot of the *same* Q8 weights (SP_ENGINE_FROB=2) agree to float-associativity ($\max |\Delta \logit| < 1e-2$; measured $\sim 1e-4$). (b) **Q8 quality** vs the engine's own pure-f32 path (NOT ggml, §8.6.1): mean $KL(f32 \parallel q8)$ below a regression gate (default $5e-2$; measured $\sim 2e-2$). Per-row Frobenius Q8 is lossy by design (one scale per wide row, vs ggml's per-32-block Q8_0), so it legitimately flips a few low-margin argmaxes — argmax is reported, not gated. Real PPL quality is T_FRO_4.
 - E_CPU_4 — AVX2 matmul (8-wide FMA, dot_f32) vs the scalar reference (SP_CPU_SCALAR=1): argmax agreement at every position + $\max |\Delta \logit|$ below the float-reassociation floor (default $1e-3$; measured $\sim 1.7e-4$). The original "1e-6 elementwise" is unachievable on *final* logits — FMA and 8-wide accumulation reassociate the dot, and QK-RMSNorm amplifies that over 28 layers exactly as in §8.6.1 (1e-6 would only hold per single matmul output). AVX512 shares the gate when built.
 - E_CPU_5 — NTT-attention (SP_ENGINE_NTT_ATTN=1, sieve OFF): each attention score $\langle q, k \rangle$ is recovered EXACTLY as coefficient 0 of the negacyclic poly-ring product (sp_pr_inner , $head_dim=128=ring\ N$) after int32 quantization (scale 2^{16}) of the post-norm/post-RoPE head vectors; softmax + V-sum stay f32. Gate: argmax agreement + mean end-to-end $KL(softmax-baseline \parallel ntt) \leq 1e-7$ (the literal T_PR_2). Measured $\sim 2.7e-10$ mean — because the inner product is exact (only int32 quant deviates), this meets the tight T_PR_2 bound end-to-end, unlike the F16/FMA gaps of E_CPU_2/E_CPU_4.
 - E_CPU_6 — KSTE KV-cache overlay (qwen3_forward_ex with a kv_trees sink; production gate SP_KSTE_KV=1), sieve OFF, encoder only. KSTE is a one-way deterministic encoder (no decode), so the "round-trip identical bytes" is byte-identical determinism + store/load: each post-norm/post-RoPE K head-vector (int32-quantized) is encoded to its 64-byte signature, and the test asserts (1) two independent prefills produce byte-identical signatures, (2) every signature survives a store/load unchanged and carries the frozen wire form (version/branching/depth/reserved + $k=head_dim$), (3) distinct K vectors give distinct signatures. Measured on Qwen3-0.6B: 6944 signatures

(28×31×8), all deterministic + wire-valid.

- **Notes for picking up.** CPU is the canonical track because it has the fewest build dependencies. Bring up CPU first if any other backend is blocked; using CPU outputs as the ground truth for the other backends is built into the test harness.

T_FRO_4 — Gemma3-1B PPL gate (CLOSED 2026-05-22, SP3 commit 3431cf8). The §6 / T4 acceptance check, recomputed under the production code. The original single “PPL within 0.1% of baseline” target conflated two independent things and is **superseded** by a split gate — per-row Frobenius Q8 is ~1% lossy *by design* (E_CPU_3, one scale per wide row vs ggml’s per-32-block Q8_0), so 0.1% was never the right bound for the quantised path:

- **(a) forward correctness** — engine pure-f32 PPL vs the stock-llama.cpp f16 oracle on the same corpus + n_ctx, gated at the §8.6.1 precision floor (SP_PPL_GATE_F32, ≤ 0.05% rel). This is the real “Gemma3-1B forward correct” check. Measured **−0.0146%** (engine 32.865 vs oracle 32.869).
- **(b) per-row Q8 arena quality** — Q8 PPL drift vs the engine’s own f32 PPL (SP_PPL_GATE_Q8, ≤ 2% rel; judged against the engine’s f32, never ggml, per §8.6.1). Measured **−0.74%**.

The Q8 pass reloads via the packed arena (SP_ARENA=q8, byte-identical to SP_ENGINE_FROB=1 per E_CPU_9 but quantised once at load, so the gate runs in minutes); the f32 pass clears all matmul knobs first so a contaminated shell cannot silently turn it into a q8-vs-q8 false PASS. sp_perplexity follows perplexity.cpp: non-overlapping n_ctx chunks, per-chunk BOS re-anchor, score [n_ctx/2, n_ctx-1], PPL = exp(mean NLL); the single-window fixture (n_ctx=168 == token count) matches the dump_logits oracle exactly. SLOW-labelled (its own ctest run, ~158 s); the 176 MB regenerable oracle .ref.bin is gitignored. Full regression **20/20 green incl. T_FRO_4**. This clause is the authoritative T_FRO_4 gate; it supersedes the original single-number “within 0.1%” target (the §6 T4 / §7.5 figure).

8.2.1 Foundational-compression extension (amended 2026-05-22) The original six E_CPU items closed 2026-05-22 (lat-phase-2-closed). The inline weight + KV compression that §1/§4.8/§4.9 call **foundational** is only partly covered by them: E_CPU_3 is Q8 weights, E_CPU_6 is the *KSTE* KV overlay (a Lattice-sieve signature encoder, one-way) — neither is **Q4 weight storage** nor the **VHT2+Spinor KV codec** that §4.5/§4.9 freeze as the production KV layout. This amendment adds the missing foundational items as explicit Phase-2-CPU contract gates, plus the persistent-KV decode loop. All are gated OFF by default (regression invariant: gates off ⇒ bit-identical to the E_CPU_2 f32 path). Per §8.6.1 every quality gate is measured against the engine’s own pure-f32 logits, never ggml and never the F16-act path.

- **E_CPU_7 — Q4 inline weight compression.** Frobenius Q4: per-row scale, symmetric 4-bit codes in [-7,+7] (the Q8 [-127,127] rule at 4 bits; two codes per byte), dequant $w_{\text{hat}} = q \cdot s / 7$. **Mixed-precision / calibration:** a per-tensor pass promotes high-error (“outlier”) rows to Q8. v1 calibration is **weight-only per-row sensitivity** (promote rows whose Q4 round-trip error / row-L2 exceeds a threshold) — the **activation-based** calibration of §4.4 stays a **Phase-4** refinement (§7.5), with the promotion-mask hook left in place. Gate SP_ENGINE_FROB=3. Validated on Qwen3-0.6B: lift faithfulness (inline-Q4 matmul == dequant-then-f32-dot of the same Q4 weights, float-assoc) + Q4 quality vs pure-f32 (mean KL under a regression gate, looser than Q8 since Q4 is lossier by design; argmax reported, not gated).
- **E_CPU_8 — Inline VHT2+Spinor KV-cache compression.** The §4.5 frozen 63-byte Spinor block carries 55 int8 anchors; a Qwen3 head vector is head_dim=128, so each post-norm/post-RoPE K and V head vector is stored as [128/55] = **3 Spinor blocks** (balanced 43/43/42 split; the frozen layout is NOT modified). Gate SP_KV_SPINOR=1; attention reads the decoded (lossy) K'/V'. Gate OFF ⇒ bit-identical to E_CPU_2. ON: mean KL(f32-KV || spinor-KV) under a regression gate, argmax reported. (Distinct from E_CPU_6’s KSTE

overlay — this is the foundational lossy KV codec, KSTE is the sieve signature.)

- **GEN_KV — persistent-KV $O(n)$ decode loop.** A KV cache stores position-finalized (post-RoPE) K/V so decode steps append one token without re-rotating. Gate: **argmax-identity** with the $O(n^2)$ qwen3_generate (greedy sequence equals greedy sequence) under SP_CPU_SCALAR=1 — NOT bit-equal logits, because the incremental and re-prefill paths reduce different-length softmax sums and so legitimately differ by the float-reassociation floor (§8.6.1). Composes with SP_KV_SPINOR (the cache may hold Spinor blocks). Token count SP_GEN_KV_N (default 8).

8.2.2 Load-bearing layout: packed-weight arena + persistent KV (amended 2026-05-

22) E_CPU_3/7/8 prove the codecs but ran them as a per-matmul *demonstration* (re-quantize on the fly; KV stored f32 with a lossy round-trip). §4.8/§4.9 require the compression to be the **actual memory layout** — and the GPU backends must read the *same packed bytes* (§4.8 “compare bytes for bytes”), so the packed formats are built and frozen on the canonical CPU track **before** 2-CU. The Q4 quant/pack primitive was first lifted into core/frobenius (shared by all backends; math-core T_FRO_Q4). Three pieces, dependency-ordered:

- **E_CPU_9 — packed-weight arena (Piece 1a, DONE).** SP_ARENA=q8|q4 (default off): at load, quantize the matmul weights once into a per-row Frobenius Q8/Q4 packed arena; the matmul lifts inline from the codes. Versioned in-memory layout (SP_ARENA_LAYOUT_VERSION=1) = the byte format the GPU backends read (per-ROW Frobenius, NOT ggml per-32-block Q8_0). Tight gate: SP_ARENA=q8 forward **byte-identical** to SP_ENGINE_FROB=1 (and q4 to =3). Measured: Q8 arena 574.5 MB, Q4 arena 300.7 MB (2.85% rows promoted). Scope 1a: matmul weights only; embedding+norms stay f32 from the mapping (still held).
- **E_CPU_10 — release the F16 source (Piece 1b, DONE).** SP_ARENA_EMBED=1 folds the embedding into the arena; qwen3_release_source() copies norms to owned f32 and gguf_release_data() unmaps the GGUF data (keeping the parsed tensor/kv structs). The forward then reads only the arena + owned norms — peak memory drops from the ~1.5 GB F16 mapping to the packed footprint (~574 MB Q8). SP_ARENA_RELEASE=1 does it at load. Tokenizer owning mode (sp_tokenizer_load_ex(g,1)) copies vocab+merges so it survives the unmap. Gate: forward after release **byte-identical** to held (a dangling pointer would diverge/crash) + gguf_tensor_data NULL post-release + owning tokenizer still decodes. **T_FRO_4 is now CLOSED (see the §8.2 closure clause)** — the arena is the production layout it validates (Gemma3-1B PPL via the split gate; the 2nd arch + SentencePiece it needed both landed in SP2/SP3).
- **Piece 2 — persistent Spinor KV cache (DONE).** qwen3_generate_kv with SP_KV_SPINOR=1 now stores the cache as sp_spinor_block_t[] (NBLK = ceil(head_dim/55) blocks/head; 3 for Qwen3) and decodes on read into a per-LAYER f32 scratch — resident KV memory is the packed blocks + one layer of scratch, not the full f32 cache. decode(encode(x)) is arithmetically the same as the in-place round-trip, so the block cache is **sequence-identical** to the f32 round-trip parity reference, exposed as SP_KV_SPINOR_REF=1 (the §4.9 “fp32 cache for parity tests only”). **Gate GEN_KV_SPINOR** (in test_gen_kv) asserts that identity. **Measured drop: 2.71×** (block 2.96 MB vs f32 8.03 MB on Qwen3-0.6B, P=44) — the honest figure for the *frozen* 63-byte / 55-anchor block at head_dim=128 (3 blocks). The earlier “~6×” was aspirational; the frozen layout gives 512 B → 189 B per head = 2.71×. Larger drops would require changing the frozen block (out of scope; would bump SP_SPINOR_LAYOUT_VERSION).
- **Piece 3 — composability + format-lock (DONE).** COMPOSE gate (test_compose): with the arena ACTIVE (SP_ARENA=q8), the Spinor-block KV cache is still sequence-identical to the f32 KV reference — i.e. the weight source (arena Q8) and the KV layout (blocks vs f32) are orthogonal axes that compose without interference (all three gates on: SP_ARENA + SP_KV_SPINOR + qwen3_generate_kv). **Format-lock:** file-scope Static_asserts freeze the cross-backend byte contract — arena: SP_FROB_ARENA_LAYOUT_VERSION==1,

Q8 qmax 127 / Q4 qmax 7 dequant convention, 4-byte f32 row scale; KV: 63-byte block, 55-anchor body, SP_SPINOR_LAYOUT_VERSION==1. A silent layout drift is now a compile error until the version is bumped with a migration.

- **Piece 4 — port the load-bearing primitives into the math core (DONE, 2026-05-22; user direction “more load-bearing in math-core before 2-CU”).** The packed-weight arena FORMAT and the multi-block KV head split were initially written in the engine (src/forward/{arena, forward}.c); since they are cross-backend byte contracts (every backend reads the same bytes, §4.8/§4.9), they were ported into shannon-prime-system so CUDA/Vulkan/Hexagon share one implementation rather than re-deriving them. **core/frobenius** (math-core commit 9a4e0ea): sp_frob_packed_tensor (the mixed-precision per-row Q8/Q4 arena layout) + sp_frob_pack_tensor(rows, cols, prec, promote, get_row_cb, ctx, out, *promoted) (callback-based so GGUF reading stays engine-side, no full-tensor f32 spike) + sp_frob_packed_dequant_row + SP_FROB_ARENA_LAYOUT_VERSION (the single source of truth — the engine’s duplicate macro was deleted). Gate T_FRO_5. **core/vht2**: sp_spinor_blocks_for / sp_spinor_encode_vec / sp_spinor_decode_vec (the frozen 43/43/42 head split). Gate T_VHT_7 (split is byte-identical to an explicit per-chunk encode/decode). Engine (b00fbf1) bumps the submodule and consumes them; behavior byte-identical (CPU regression 16/16).

The load-bearing layout (arena + persistent Spinor KV + format-lock), and the primitives that realize it, now live in the math core; all four backends will share one implementation. 2-CU may start (user direction 2026-05-22) — its kernels read the SP_FROB_ARENA_LAYOUT_VERSION=1 arena bytes and the SP_SPINOR_LAYOUT_VERSION=1 KV blocks directly. T_FRO_4 (Gemma3-1B PPL) is the production-quality validation of the arena — CLOSED 2026-05-22 (SP3 commit 3431cf8); see the §8.2 closure clause.

8.3 Phase 2-CU (CUDA backen